```
package com.polarbookshop.edgeservice.web;

import reactor.core.publisher.Mono;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.RouterFunctions;
import org.springframework.web.reactive.function.server.ServerResponse;

@Configuration
public class WebEndpoints {

  @Bean                                                              ❶
  public RouterFunction<ServerResponse> routerFunction() {
    return RouterFunctions.route()                                   ❷
      .GET("/catalog-fallback", request ->                           ❸
        ServerResponse.ok().body(Mono.just(""), String.class))
      .POST("/catalog-fallback", request ->                          ❹
        ServerResponse.status(HttpStatus.SERVICE_UNAVAILABLE).build())
      .build();                                                      ❺
  }
}
```

❶ Functional REST endpoints are defined in a bean.

❷ Offers a fluent API to build routes

❸ Fallback response used to handle the GET endpoint

❹ Fallback response used to handle the POST endpoint

❺ Builds the functional endpoints

For simplicity, the fallback for `GET` requests returns an empty string, whereas the fallback for `POST` requests returns an HTTP 503 error. In a real scenario, you might want to adopt different fallback strategies depending on the context, including throwing a custom exception to be handled from the client or returning the last value saved in the cache for the original request.

So far, we have used retries, timeouts, circuit breakers, and failovers (fallbacks). In the next section, I'll expand on how we can work with all those resilience patterns together.